

Mithridatium — Defenses Overview

Mithridatium provides four backdoor detection defenses for pretrained image classification models. Each defense targets a different signal and involves different tradeoffs in speed, sensitivity, and architectural requirements.

Quick reference

Defense	Signal	Speed	Dataset required	Architecture	Threshold
MMBD	Class-specific activation anomaly	Medium	No	Any	p-value < 0.05
STRIP	Prediction entropy under perturbation	Fast	Yes	Any	entropy_mean > 0.45
AEVA	Adversarial boundary asymmetry	Slow	Yes	Any	anomaly_index ≥ 4.0
FreeEagle	Embedding-space routing anomaly	Medium	No	ResNet only	anomaly_metric ≥ 2.0

MMBD

Multi-Model Backdoor Detection optimizes synthetic inputs toward each class and measures whether one class attracts anomalously strong activation compared to the rest. A backdoored model produces a statistically dominant response for its target class, detected via a gamma-distribution p-value test.

- Verdict field: "Likely clean" / "Likely backdoored"
- Key metric: `p_value` (threshold: 0.05) and `normalized_scores` (flag if any score > 5.0)
- Does not require dataset access
- Probes 5 classes per run by default — if the backdoor target falls outside the probed set it may be missed

[Full documentation →](#)

STRIP

STRong Intentional Perturbation mixes base images with random images and measures prediction entropy over the perturbed inputs. In this implementation, a backdoored model produces higher and more variable entropy than a clean model under perturbation.

- Verdict field: "likely clean" / "likely backdoored"
- Key metric: `entropy_mean` (default threshold: 0.45)
- Fastest defense — forward passes only, no optimization

- The default threshold is not well-calibrated for all datasets; inspect `entropy_std` alongside `entropy_mean`

[Full documentation →](#)

AEVA

Adversarial Example-based Vulnerability Analysis runs a black-box boundary attack (HSJA) across all source-target class pairs and measures the concentration of adversarial perturbations per target class. A backdoored model has one class that attracts concentrated, low-effort perturbations from all others.

- Verdict field: `"likely clean"` / `"likely backdoored"`
- Key metric: `suspicion_score` (anomaly_index threshold: 4.0); also reports `suspected_target`
- Slowest defense — scales as $O(\text{num_classes}^2)$ model queries; GPU strongly recommended
- Caches perturbation results to disk; repeated runs on the same model are fast
- Supported datasets: `cifar10` and `cifar100` only

[Full documentation →](#)

FreeEagle

FreeEagle optimizes dummy embeddings for each class in the model's intermediate feature space and measures whether any class disproportionately attracts softmax mass from all others. Detection requires no input data — only the model weights.

- Verdict field: `"likely clean"` / `"likely backdoored"`
- Key metric: `anomaly_metric` (IQR-based outlier score, threshold: 2.0)
- Does not require dataset access
- ResNet-family models only
- Inspect `tendency_per_target` to identify the suspected target class (not reported automatically)

[Full documentation →](#)

Choosing a defense

Start with STRIP if you need a fast first pass. It requires only forward passes and gives an immediate signal, though its threshold needs calibration.

Use MMBD or FreeEagle when you do not have dataset access, or when you want a statistically grounded result without running adversarial attacks. MMBD provides a p-value; FreeEagle provides an IQR-based outlier score.

Use AEVA when you want the strongest evidence and have GPU access and time. It is the most computationally intensive but also the most direct — it measures actual decision boundary asymmetry rather than a proxy signal, and it identifies the suspected target class.

CLI

All defenses are run through the same `detect` command:

```
mithridatium detect \  
  --model  models/resnet18.pth \  
  --data   cifar10 \  
  --defense [mmbd | strip | aevea | freeeagle] \  
  --out    reports/report.json
```

See each defense's documentation for defense-specific CLI flags.

Exit codes

Code	Meaning
64	Invalid CLI usage (e.g., unsupported <code>--defense</code>)
66	Model path missing or not a file
73	Output file exists and <code>--force</code> not supplied
74	I/O error (model load failed, report schema invalid)